

1-Mavzu: “Sun’iy intellekt asoslari” faniga kirish va asosiy tushunchalari.
10-mashg’ulot. Pythonda Funksiya tushunchasi. Foydalanuvchi funksiyasi.
O‘quv savollari:

1. Funksiyalarni aniqlash va uni chaqirish.
2. Parametrli va parametrsiz funksiya.

1-o‘quv savoli: Pythonda funksiyaning aniqlash va uni chaqirish

1) Funksiya nima?

Funksiya — bu bir marta yozib, ko‘p marta ishlatadigan kod blokidir.

Asosiy foydasi:

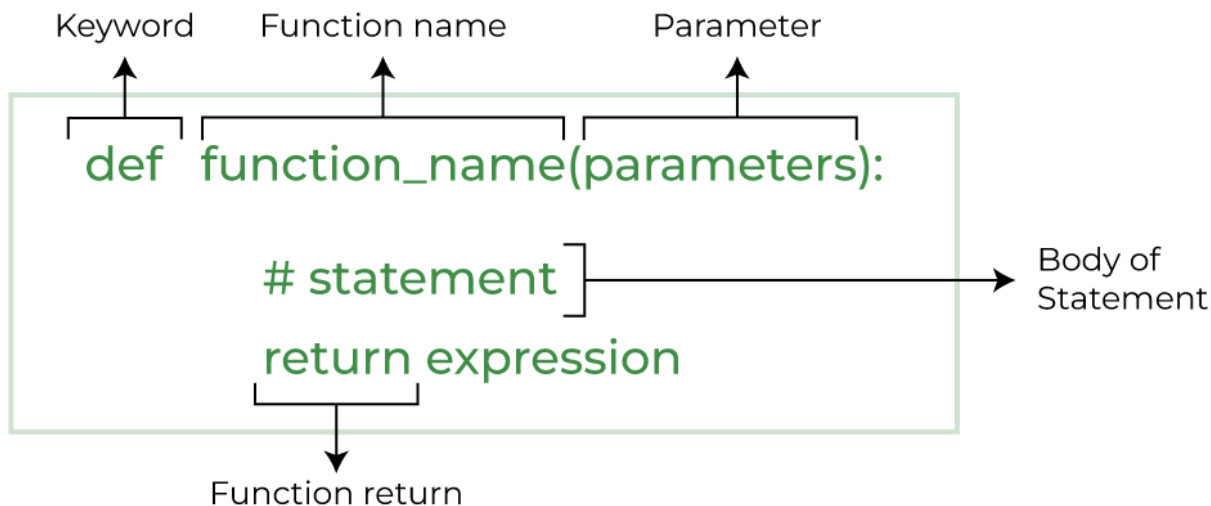
- kodni **tartibli** va **qayta foydalanish** mumkin qiladi;
- murakkab masalani **kichik bo‘laklarga** bo‘ladi;
- xatoni topish va test qilishni osonlashtiradi.

Pythonda **print()**, **len()** kabi **tayyor (standart)** funksiyalardan foydalaniladi. **Foydalanuvchi funksiyasi** esa — foydalanuvchi tomonidan yoziladigan funksiya.

2) Funksiyaning aniqlash (e‘lon qilish) — def sintaksisi

```
def funksiya_nomi():  
    # bu yerda funksiyaning tanasi (body)  
    # kerakli buyruqlar yoziladi  
    # ixtiyoriy: natija qaytarish uchun return ishlatiladi  
    ...
```

- **def** — **aniqlash** kalit so‘zi
- **funksiya_nomi** — odatda **snake_case**: hisobla_summa, print_holat
- **()** — parametrlar joyi (bo‘sh bo‘lsa — parametrsiz funksiya)
- **:** — yangi blok boshlanishi
- **Ichidagi kod** to‘g‘ri **indentatsiya** (odatda 4 ta probel) bilan yoziladi
- **return** — funksiya **natija** qaytaradi (ixtiyoriy). **return** bo‘lmasa, funksiya **None** qaytaradi.



Eng oddiy misol (parametrsiz)

```
def salom():  
    print("Salom, dunyo!")
```

3) Funksiyani chaqirish

Funksiyani ishga tushirish — **chaqirish** deyiladi.

Sintaksis:

```
funksiya_nomi()
```

agar parametrlar bo'lsa: `funksiya_nomi(arg1, arg2, ...)`.

```
salom()          # Salom, dunyo!  
salom()          # Salom, dunyo!
```

Muhim: `salom` (qavssiz) — bu funksiya **obyektiga ishora**.

`salom()` — **chaqirish** (ishga tushirish). Qavsni unutish — keng tarqalgan xato.

4) `print` va `return` farqi

- `print(...)` — natijani ekranga chiqaradi, lekin **qiymat qaytarmaydi**.
- `return qiymat` — **qiymat qaytaradi** (keyin undan o'zgaruvchiga saqlab foydalanish mumkin).

```
def kvadrat_print(n):  
    print(n * n)          # faqat chiqaradi  
  
def kvadrat_return(n):  
    return n * n          # qiymat qaytaradi  
  
kvadrat_print(5)         # ekranga 25 yozadi  
x = kvadrat_return(5)  
print("x =", x)          # x = 25
```

5) Qachon `return` kerak bo'ladi?

- Natijani keyinroq **hisob-kitoblarda** ishlatmoqchi bo'lsangiz.
- Funksiya **qiymat hisoblab** berishi kerak bo'lsa (masalan, matematik funksiya).

```
def uchburchak_yuzi(a, h):  
    return (a * h) / 2  
  
S = uchburchak_yuzi(10, 6)  
print(S)          # 30.0
```

`return` bajarilgach, funksiya **shu zahoti tugaydi** — undan keyingi satrlar ishlamaydi.

6) Ko'p qiymat qaytarish

Python bitta return bilan **bir nechta qiymat** qaytara oladi (tuple):

```
def min_max(sonlar):
    mn = min(sonlar)
    mx = max(sonlar)
    return mn, mx

m, M = min_max([3, 9, 1, 7])  # ajratib olish (unpacking)
print(m, M)                  # 1 9
```

7) Docstring (ixtiyoriy, lekin foydali)

Docstring — bu funksiya boshida yoziladigan matnli izoh. U funksiya nima qilishini qisqa tushuntiradi va `help()` orqali ko'rsatib beradi.

```
def salom_ber(ism):
    """Foydalanuvchi bilan salomlashadi."""
    print(f"Salom, {ism}!")
help(salom_ber)  # docstring chiqadi
```

Ya'ni docstring — funksiya maqsadini qisqacha yozib qo'yish uchun ishlatiladi.

```
salom_ber(ism)
Foydalanuvchi bilan salomlashadi.
```

8) Type hint (o'qishni osonlashtiradi)

Type hint — bu parametr va natijaning turini ko'rsatish. Kodni tushunish va o'qishni osonlashtiradi.

Misol:

```
def kuch(a: int, n: int) -> int:
    """a ning n-darajasi"""
    return a ** n
```

Bu yerda `a` va `n` butun son (`int`), natija ham `int`.

9) Kod bajarilish tartibi (Execution order)

Python har doim kodni yuqoridan pastga qarab ketma-ket bajaradi.

Shuning uchun:

- Funksiya **avval aniqlanib**, keyin chaqirilishi kerak.
- Agar aniqlashdan oldin chaqirilsa, `NameError` chiqadi.

Misol:

```
def f():
    print("OK")

f()  # ishlaydi
```

```
g() # xato: funksiya hali aniqlanmagan
def g():
    print("Hello")
```

Amaliy mashqlar:

Savol 1. (print va return farqi)

Do‘kon savdo dasturida `hisobla()` funksiyasini yozing: u berilgan mahsulot narxi va sonini qabul qiladi.

- Birinchi variant faqat **ekranga chiqarishi** kerak.
 - Ikkinchi variant esa **qiymat qaytarishi** kerak.
- Farqini tushuntiring.

Javob:

```
# faqat chiqaradigan funksiya
def hisobla_print(narx, son):
    print("Umumiy summa:", narx * son)

# qiymat qaytaradigan funksiya
def hisobla_return(narx, son):
    return narx * son

hisobla_print(5000, 3) # "Umumiy summa: 15000"

umumiy = hisobla_return(5000, 3)
print("Chekda ko'rsatilgan summa:", umumiy) # 15000
```

Birinchisi faqat chiqaradi, keyin boshqa joyda ishlatib bo‘lmaydi. Ikkinchisi esa qiymatni saqlab, boshqa hisob-kitobda ham ishlatish mumkin.

Savol 2.

Sportchilar ro‘yxatidan eng tez va eng sekin yugurganni topadigan `rekord()` funksiyasini yozing.

Javob:

```
def rekord(vaqtlar):
    mn = min(vaqtlar)
    mx = max(vaqtlar)
    return mn, mx

min_vaqt, max_vaqt = rekord([12.5, 11.8, 13.0, 10.9])
print("Eng tezkor:", min_vaqt)
print("Eng sekin:", max_vaqt)
```

Funksiya **tuple** qaytaradi, biz esa natijani alohida o‘zgaruvchilarga ajratib olamiz.

Savol 3. (Docstring va help)

Bank hisobidan pul yechadigan `pul_yech()` funksiyasini yozing va unga docstring qo'shing. Keyin `help()` orqali natijani ko'rsating.

Javob:

```
def pul_yech(balans, summa):  
    """Balansdan summa yechib, qolganini qaytaradi."""  
    if summa > balans:  
        return "Xatolik: balans yetarli emas"  
    return balans - summa  
  
help(pul_yech)  
  
print(pul_yech(1000, 300))    # 700
```

Savol 4. (Execution order)

Quyidagi kod nima uchun xato beradi? To'g'ri variantini yozing.

```
hisob()    # Xato: aniqlashdan oldin chaqirilgan  
  
def hisob():  
    print("Hisoblash boshlandi")
```

To'g'ri variant:

```
def hisob():  
    print("Hisoblash boshlandi")  
  
hisob()    # endi ishlaydi
```

Python kodni yuqoridan pastga bajaradi, shuning uchun funksiya **oldin e'lon qilinishi kerak**.

2-o'quv savoli. Parametrli va parametrsiz funksiya.

Parametrsiz funksiya

Ta'rif: Parametrsiz funksiya — chaqirilganda tashqaridan qiymat olmaydigan, o'z ichki ma'lumotlari (yoki global ma'lumotlar) bilan ishlaydigan funksiya.

Sintaksis:

```
def salom():  
    """Oddiy salomlashish."""  
    print("Salom, dunyo!")
```

Chaqirish:

```
salom() # "Salom, dunyo!" chiqadi
```

Parametrsiz funksiya qachon kerak?

1. Bir xil ishni takror-takror bajarish uchun

Masalan, konsolda chiziq chizish, menyu chiqarish, yoki har safar bir xil xabarni ko'rsatish.

```
def ajratkich():  
    print("-" * 30)  
  
def menyu():  
    print("1) Boshlash")  
    print("2) Sozlamalar")  
    print("3) Chiqish")  
  
# Har safar kerak bo'lganda chaqiramiz:  
ajratkich()  
menyu()  
ajratkich()
```

Bu yerda hech qanday parametr kiritish shart emas, chunki funksiya **doim bir xil** narsani qiladi.

```
-----  
1) Boshlash  
2) Sozlamalar  
3) Chiqish  
-----
```

2. Test yoki demo maqsadlarida

Yangi dastur yozayotganda, kodni sinash yoki misol ko'rsatish uchun oddiy funksiyalar kerak bo'ladi.

```
def salom():  
    print("Salom, dunyo!")
```

```
# Test qilamiz:
salom() # "Salom, dunyo!"
salom() # yana chiqaradi
```

Bu funksiyaning foydasi — dastur ishlayotganini ko'rsatish

Xulosa: **Parametrsiz funksiya** — har safar **bir xil natija** kerak bo'lganda yoki **oddiy misol/test** uchun qulay.

Parametrli funksiya

Ta'rif: Parametrli funksiya — chaqirilganda tashqaridan **argument(lar)** qabul qiladi va shu qiymatlar bilan ishlaydi.

Oddiy misol:

```
def uchburchak_yuzi(a, h):
    """Asos a va balandlik h bo'yicha yuzani qaytaradi."""
    return (a * h) / 2

S = uchburchak_yuzi(10, 6) # 30.0
```

Argumentlarni berish usullari

1. Pozitsion (tartib bo'yicha):

```
print(pow(2, 5)) # 2**5: 32
```

2. Kalit so'z (keyword) orqali:

```
print(pow(x=2, y=5)) # o'qish aniqroq
```

3. Aralash:

```
pow(2, y=5) # OK: avval pozitsion, keyin keyword
```

!!! Kalit so'zdan keyin **yana pozitsion** argument berib bo'lmaydi.

Standart qiymatlar

Parametrlarni ixtiyoriy qilish uchun **standart qiymat** berish:

```
def salom(ism="dunyo"):
    print(f"Salom, {ism}!")

salom() # "Salom, dunyo!"
salom("Ali") # "Salom, Ali!"
```

Tavsiya: Majburiy parametrlar **chapda**, ixtiyoriylari **o'ngda** tursin:

```
def connect(host, port=5432, timeout=5):
```

...

None orqali ixtiyoriy parametrlar

Ba'zan "berilsa-berilsin, berilmasa-berilmasin" mantiqi kerak bo'ladi:

```
def greet(name=None):  
    if name is None:  
        print("Salom!")  
    else:  
        print(f"Salom, {name}!")
```

Ko'p sonli argumentlar: *args, **kwargs

- *args — **cheklanmagan** pozitsion argumentlar tuple ko'rinishida keladi.
- **kwargs — **cheklanmagan** keyword argumentlar dict ko'rinishida keladi.

```
def summa(*args):  
    return sum(args)  
  
def info(**kwargs):  
    # masalan: ism="Ali", yosh=20  
    for kalit, qiymat in kwargs.items():  
        print(kalit, "→", qiymat)  
  
print(summa(1, 2, 3, 4)) # 10  
info(ism="Ali", yosh=20)
```

Python'da funksiya **parametrlarni belgilash tartibida** aniq qoida bor. U quyidagicha:

Parametrlarni belgilash tartibi: pozitsion → standart → *args → keyword-only → **kwargs.

1. Pozitsion parametrlar

Bular **majburiy** bo'lgan oddiy parametrlar. Chaqirilganda tartib bo'yicha beriladi.

```
def f(a, b):  
    print(a, b)  
  
f(1, 2) # a=1, b=2
```

2. Standart (default) parametrlar

Agar qiymat berilmasa, oldindan belgilangan qiymat ishlatiladi.

Majburiy (pozitsion) parametrlar har doim oldinda turadi.

```
def f(a, b=10):  
    print(a, b)
```



```
f(5)          # a=5, b=10 (standart)
f(5, 7)       # a=5, b=7
```

3. *args (cheklanmagan pozitsion argumentlar)

Qancha pozitsion qiymat berilsa, hammasi **tuple** ko'rinishida `*args` ichiga tushadi.

```
def f(a, *args):
    print("a =", a)
    print("args =", args)

f(1, 2, 3, 4)
# a=1, args=(2, 3, 4)
```

4. Keyword-only parametrlar

Agar parametr `*` dan keyin yozilsa, uni faqat **nomi bilan** berish kerak.

```
def f(a, *, b, c=0):
    print(a, b, c)

f(1, b=2)          # OK
# f(1, 2)          # ✗ Xato: b ni nom bilan yozish shart
```

5. **kwargs (cheklanmagan keyword argumentlar)

Qo'shimcha kalit=qiymat ko'rinishidagi argumentlar **dict** sifatida yig'iladi.

```
def f(**kwargs):
    print(kwargs)

f(ism="Ali", yosh=20)
# {'ism': 'Ali', 'yosh': 20}
```

6. Parametr turlarining to'liq kombinatsiyasi

```
def demo(a, b=0, *args, c, **kwargs):
    print("a =", a)
    print("b =", b)
    print("args =", args)
    print("c =", c)
    print("kwargs =", kwargs)

demo(1, 2, 3, 4, 5, c=6, x=7, y=8)
```

Natija:

```
a = 1          # pozitsion
b = 2          # standart (default)
args = (3, 4, 5) # *args
```

```
c = 6 # keyword-only
kwargs = {'x': 7, 'y': 8} # **kwargs
```

Xulosa:

Funksiya parametrlarini e'lon qilishda tartib **qattiq**:

1. Oddiy pozitsionlar
2. Standart (default) parametrlar
3. `*args`
4. Keyword-only (agar bo'lsa)
5. `**kwargs`

Python funksiyadagi parametrlar tartibi

T/r	Parametr turi	Sintaksis ko'rinishi	Xususiyati
1	Pozitsion (majburiy)	<code>a, b</code>	Argumentlar ketma-ketlik bo'yicha beriladi.
2	Standart (default)	<code>b=0, c="hi"</code>	Agar qiymat berilmasa, sukut (default) qiymat ishlatiladi.
3	Cheklanmagan pozitsion	<code>*args</code>	Bir nechta qo'shimcha pozitsion argumentlarni tuple sifatida yig'adi.
4	Keyword-only	<code>*, d, e=1</code>	<code>*</code> dan keyin kelgan parametrlar faqat nomi bilan berilishi kerak.
5	Cheklanmagan keyword	<code>**kwargs</code>	Qo'shimcha <code>kalit=qiymat</code> juftliklarini dict sifatida yig'adi.

Keyword-only va positional-only parametrlar (advanced)

Python'da parametrlarni **qanday berish mumkinligini** yanada aniq belgilash uchun 2 ta maxsus belgidan foydalaniladi:

1. `*` → Keyword-only parametrlar

- `*` dan keyin yozilgan parametrlar **faqat nomi bilan** berilishi shart.
- Ularni tartib (pozitsiya) bo'yicha yozib bo'lmaydi.

Misol:

```
def salom(ism, *, xabar="Salom"):
    print(f"{xabar}, {ism}!")

salom("Ali", xabar="Assalomu alaykum") # ✓ OK
# salom("Ali", "Assalomu alaykum")    # ✗ Xato
```

`xabar` parametri **keyword-only**, shuning uchun uni `xabar="..."` tarzida yozish kerak.

2. `/` → Positional-only parametrlar (Python 3.8+)

- `/` dan oldin yozilgan parametrlar **faqat pozitsion** tarzda beriladi.
- Ularni nomi bilan yozib bo'lmaydi.

Misol:

```
def kvadrat(son, /):  
    return son * son  
  
print(kvadrat(5))          # ✓ OK  
# print(kvadrat(son=5)) # ✗ Xato
```

son parametri **positional-only**, shuning uchun uni faqat tartib bo'yicha berish mumkin.

3. Birlashtirish: / va * ni birga ishlatish

```
def f(a, b, /, c, *, d):  
    # a, b → faqat pozitsion  
    # c → pozitsion ham, keyword ham bo'lishi mumkin  
    # d → faqat keyword  
    return a + b + c + d
```

Tahlil:

- a, b — / oldida, shuning uchun **faqat pozitsion**.
- c — o'rtada, shuning uchun **ikkala usulda ham** berish mumkin.
- d — * dan keyin, shuning uchun **faqat keyword**.

Misol:

```
print(f(1, 2, 3, d=4))      # ✓ OK (to'g'ri)  
# print(f(a=1, b=2, c=3, d=4)) # ✗ Xato (a,b ni nom bilan bermaymiz)
```

Xulosa

- / → undan oldingi parametrlar **pozitsion-only**
- * → undan keyingi parametrlar **keyword-only**
- O'rtadagi parametrlar esa **ikkala usulda ham** berilishi mumkin.

Amaliy mashqlar:

Savol 1. (Parametrsiz funksiya)

Har safar chiziq chiqaradigan `ajratkich()` funksiyasini yozing. So'ng uni parametrli ko'rinishga o'tkazing, ya'ni uzunlik va belgini foydalanuvchi belgilasin.

Javob:

```
# Parametrsiz  
def ajratkich():  
    print("-" * 20)  
  
ajratkich()
```

```
# Parametrli
def ajratkich2(uzunlik=20, belgi="-"):
    print(belgi * uzunlik)

ajratkich2()          # -----
ajratkich2(10, "*")   # *****
```

Savol 2. (Pozitsion va keyword argumentlar)

uchburchak_yuzi() funksiyasini yozing. Uni pozitsion argumentlar bilan, keyword argumentlar bilan, va aralash usulda chaqiring.

Javob:

```
def uchburchak_yuzi(a, h):
    return (a * h) / 2

# pozitsion
print(uchburchak_yuzi(10, 5))

# keyword
print(uchburchak_yuzi(a=10, h=5))

# aralash
print(uchburchak_yuzi(10, h=5))
```

Savol 3. (*args va **kwargs)

hisobot() funksiyasini yozing:

- Qancha son berilsa, hammasini yig'ib chiqsin (*args).
- Foydalanuvchi haqida qo'shimcha ma'lumotlarni (**kwargs) ham chiqarsin.

Javob:

```
def hisobot(*args, **kwargs):
    print("Yig'indi:", sum(args))
    for k, v in kwargs.items():
        print(f"{k}: {v}")

hisobot(10, 20, 30, ism="Ali", yosh=25)
```

Savol 4. (Keyword-only va positional-only)

foiz_hisobla() funksiyasini yozing:

- Asosiy summa faqat **pozitsion** berilsin.
- Foiz stavkasi esa faqat **keyword** orqali berilsin.

Javob:

```
def foiz_hisobla(s, /, *, stavka):  
    return s * stavka / 100  
  
print(foiz_hisobla(1000, stavka=12)) # ✓ 120  
# print(foiz_hisobla(s=1000, stavka=12)) # ✗ Xato: s pozitsion-only
```